Each time a SYN packet is received, the TCP allocates/reserves a resource on its machine; so a flood of SYN packets by a malicious user means that the TCP server refuses connections after the limit of its data-structures is reached.

SCTP cleverly overcomes this using a four-way handshake (see Figure 6). Here, SCTP plays safe and allots resources only when the connection initiating entity (client) has established its identity (and good intentions). This is how it works:

1. Node A (client) creates an INIT message (an 'INIT *chunk*', technically), and sends it to Node B (server). It also starts an INIT-timer to handle timeout and kick-in retransmissions.
2. If Node B is open to accept connections, it generates an INIT-ACK chunk, includes a unique value in it, named 'cookie', and sends it to Node A. At this point, no resources are allotted for this particular association at Node A.
3. Now, when Node A receives the cookie, it mirrors the same in a COOKIE-ECHO chunk. An appropriate timer is started to take care of timeouts.
4. Finally, when Node B receives the ECHO chunk, it validates the cookie, sends back a COOKIE-ACK chunk, and then allots resources for the association.

So we see how SYN flooding leading to a DOS, is overcome in SCTP design. Once the four-way handshaking is done, the association is said to be in an 'established' state. Data transmission can legally begin now. In fact, the specification even allows DATA chunks to be bundled with COOKIE and COOKIE-ACK chunks.

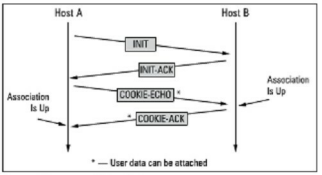

Figure 6: A four-way SCTP handshake

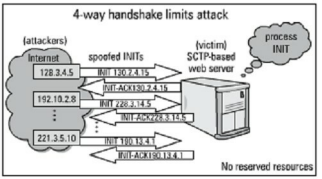

Figure 7: How SCTP overcomes DOS attacks

Hello... are you there?

Constant awareness of the peer and path status is important in SCTP. This allows it to efficiently use its multi-homing feature. This is achieved through a Heartbeat (and Ack) mechanism. A HEARTBEAT chunk is sent to any destination address that has been idle for longer than the heartbeat period. The peer responds with a HEARTBEAT-ACK to indicate that it is still alive. Let us note that Heartbeat is sent only on 'idle' addresses. On an IP that is active (i.e., DATA chunks flying here and there), Heartbeat is unnecessary: A doctor doesn't check the pulse of a patient to figure out whether he is dead or alive, if the patient is describing his ailment!

RFC 3286 describes the path failure detection mechanism as follows (go on, read it: even though it is taken from an RFC, it is surprisingly interesting): "A count is maintained of the number of retransmissions to a particular destination address without successful acknowledgement. When this count exceeds a configured maximum, the address is declared inactive, notification is given to the application, and the SCTP begins to use an alternate address for the sending of DATA chunks.

"Also, Heartbeat chunks are sent periodically to all idle destinations (i.e., alternate addresses), and a counter is maintained on the number of Heartbeats sent to an inactive destination without receipt of a corresponding Heartbeat Ack. When this counter exceeds a configured maximum, that destination address is also declared inactive.

"Heartbeats continue to be sent to inactive destination addresses until an Ack is received, at which point the address can be made active again. The rate of sending Heartbeats is tied to the RTO estimation plus an additional delay parameter that allows Heartbeat traffic to be tailored according to the needs of the user application."

And as to peer failure: "A count is maintained across all destination addresses on the number of retransmits or Heartbeats sent to the remote endpoint without a successful Ack. When this exceeds a configured maximum, the endpoint is declared unreachable, and the SCTP association is closed."

Assoc shutdown

Even in shutdown procedures, SCTP has some significant advantages over TCP. For instance, SCTP implements a graceful close of an association by exchanging three messages. These messages acknowledge that both endpoints will cease in their transmissions of data. This allows no room for 'half-open' connections (In TCP, a 'half-open' connection happens when one endpoint that continues to send data though the peer endpoint is no longer transmitting data.)